```
import matplotlib.pyplot as plt
import pandas as pd
from sklearn.cluster import AgglomerativeClustering, KMeans
from sklearn.metrics import silhouette_score
from sklearn.preprocessing import StandardScaler

# Load dataset
data = pd.read_csv("Mall_Customers.csv")

# Feature selection and scaling
X = data[["Annual Income (k$)", "Spending Score (1-100)"]]
X = StandardScaler().fit_transform(X)

# K-Means Clustering
kmeans = KMeans(n_clusters=5, random_state=42, n_init=10)
k_labels = kmeans.fit_predict(X)
print("K-Means Inertia:", round(kmeans.inertia_, 2))
print("K-Means Silhouette Score:", round(silhouette_score(X, k_labels), 3))

# Agglomerative Hierarchical Clustering
agg = AgglomerativeClustering(n_clusters=5)
a_labels = agg.fit_predict(X)
print("\nHierarchical Silhouette Score:", round(silhouette_score(X, a_labels), 3))

# Visualisation
plt.figure(figsize=(10, 4))

# Subplot 1: K-Means
plt.subplot(1, 2, 1)
plt.scatter(X[:, 0], X[:, 1], c=k_labels, cmap="viridis")
plt.title("K-Means Clustering")
plt.xlabel("Annual Income (Standardized)")
plt.ylabel("Spending Score (Standardized)")

# Subplot 2: Hierarchical
plt.subplot(1, 2, 2)
plt.scatter(X[:, 0], X[:, 1], c=a_labels, cmap="viridis")
plt.title("Hierarchical Clustering")
plt.xlabel("Annual Income (Standardized)")
plt.ylabel("Spending Score (Standardized)")

plt.tight_layout()
plt.show()
```

Polynomial regression

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures
from sklearn.metrics import mean_squared_error, r2_score

# Load dataset
data = pd.read_csv("auto-mpg.csv")

# Handle missing values
data.replace("?", np.nan, inplace=True)
data.dropna(inplace=True)

print(data.columns)

# Convert displacement to float
data["displacement"] = data["displacement"].astype(float)

# Select features and target
X = data[["displacement"]]
y = data["mpg"]

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Linear Regression
linear_model = LinearRegression()
linear_model.fit(X_train, y_train)

y_pred_linear = linear_model.predict(X_test)
linear_mse = mean_squared_error(y_test, y_pred_linear)
linear_r2 = r2_score(y_test, y_pred_linear)

# Polynomial Regression - Degree 2
poly2 = PolynomialFeatures(degree=2)
X_train_poly2 = poly2.fit_transform(X_train)
X_test_poly2 = poly2.transform(X_test)

poly_model2 = LinearRegression()
poly_model2.fit(X_train_poly2, y_train)

y_pred_poly2 = poly_model2.predict(X_test_poly2)
poly2_mse = mean_squared_error(y_test, y_pred_poly2)
```

Polynomial regression

```
poly2_r2 = r2_score(y_test, y_pred_poly2)

# Polynomial Regression - Degree 3
poly3 = PolynomialFeatures(degree=3)
X_train_poly3 = poly3.fit_transform(X_train)
X_test_poly3 = poly3.transform(X_test)

poly_model3 = LinearRegression()
poly_model3.fit(X_train_poly3, y_train)

y_pred_poly3 = poly_model3.predict(X_test_poly3)
poly3_mse = mean_squared_error(y_test, y_pred_poly3)
poly3_r2 = r2_score(y_test, y_pred_poly3)

# Compare models
comparison = pd.DataFrame({
    "Model": [
        "Linear Regression",
        "Polynomial Regression (Degree 2)",
        "Polynomial Regression (Degree 3)"
    ],
    "MSE": [
        round(linear_mse, 2),
        round(poly2_mse, 2),
        round(poly3_mse, 2)
    ],
    "R2 Score": [
        round(linear_r2, 2),
        round(poly2_r2, 2),
        round(poly3_r2, 2)
    ]
})

print(comparison)

# Generate points for plotting
X_plot = pd.DataFrame({
    "displacement": np.linspace(
        X["displacement"].min(),
        X["displacement"].max(),
        300
    )
})

X_plot_poly = poly2.transform(X_plot)
y_plot = poly_model2.predict(X_plot_poly)

# Visualize results
```

Polynomial regression

```
plt.figure(figsize=(8, 6))

plt.scatter(X["displacement"], y, color="blue", label="Actual Data")
plt.plot(
    X_plot["displacement"],
    y_plot,
    color="red",
    linewidth=2,
    label="Polynomial Regression (Degree 2)"
)

plt.xlabel("Engine Displacement")
plt.ylabel("Miles Per Gallon (MPG)")
plt.title("Polynomial Regression Fit")
plt.legend()
plt.show()
```

KNN

```
import numpy as np
import time
from sklearn.datasets import fetch_openml
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# Load Fashion-MNIST dataset
data = fetch_openml(
    'Fashion-MNIST',
    version=1,
    as_frame=False,
    parser='auto'
)

X = data.data[:2000] / 255.0
y = data.target[:2000].astype(int)

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=200, random_state=42
)

# KNN function
def knn(X_train, y_train, x, k):
```

Polynomial regression

```
    distance = np.sqrt(np.sum((X_train - x) ** 2, axis=1))
    nearest = np.argsort(distance)[:k]
    labels = y_train[nearest]
    return np.bincount(labels).argmax()

# Test different K values
for k in [3, 5, 7]:
    start = time.time()

    predictions = []

    for x in X_test:
        predictions.append(knn(X_train, y_train, x, k))

    accuracy = accuracy_score(y_test, predictions)
    end = time.time()

    print("K =", k)
    print("Accuracy =", accuracy)
    print("Time =", round(end - start, 2), "seconds")
    print()
```

ID3

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import accuracy_score, classification_report

# Load dataset
data = pd.read_csv(r'OnlineRetail.csv', encoding="ISO-8859-1")
print("Data columns:", data.columns)

# Remove missing values
data = data.dropna()

# Calculate total amount
data["TotalAmount"] = data["Quantity"] * data["UnitPrice"]
print("Data columns:", data.columns)

# Customer aggregation
customer = data.groupby("CustomerID").agg({
    "Quantity": "sum",
```

Polynomial regression

```
    "TotalAmount": "sum",
    "InvoiceNo": "nunique"
}).reset_index()

customer.columns = [
    "CustomerID", "TotalQuantity", "TotalSpent", "PurchaseFrequency"
]

# Create customer segments
customer["Segment"] = (
    customer["TotalSpent"] > customer["TotalSpent"].median()
).astype(int)

# Features and target
X = customer[["TotalQuantity", "PurchaseFrequency", "TotalSpent"]]
y = customer["Segment"]

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Train Decision Tree
model = DecisionTreeClassifier(
    criterion="entropy", max_depth=3, random_state=42
)
model.fit(X_train, y_train)

# Prediction
y_pred = model.predict(X_test)

# Evaluation
print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nClassification Report")
print(classification_report(y_test, y_pred))

# Feature importance
print("\nFeature Importance")
for feature, importance in zip(X.columns, model.feature_importances_):
    print(feature, ":", round(importance, 3))

# Visualize Decision Tree
plt.figure(figsize=(12, 8))
plot_tree(
    model,
    feature_names=X.columns,
    class_names=["Low Value", "High Value"],
    filled=True
```

Polynomial regression

```
)
plt.show()
```

Svm

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score

iris = load_iris()
X = iris.data[:, :2]
y = iris.target

y = np.where(y == 0, 1, -1)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model = SVC(kernel='linear', C=1)
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))

w = model.coef_[0]
b = model.intercept_[0]

print("Weights:", w)
print("Bias:", b)

plt.figure(figsize=(8, 6))

plt.scatter(
    X_train[y_train == 1, 0],
    X_train[y_train == 1, 1],
    label="Setosa"
)

plt.scatter(
    X_train[y_train == -1, 0],
```

Polynomial regression

```
    X_train[y_train == -1, 1],
    label="Non-Setosa"
)

x_values = np.linspace(X[:, 0].min(), X[:, 0].max(), 100)

y_boundary = -(w[0] * x_values + b) / w[1]
y_margin1 = -(w[0] * x_values + b - 1) / w[1]
y_margin2 = -(w[0] * x_values + b + 1) / w[1]

plt.plot(x_values, y_boundary, label="Decision Boundary")
plt.plot(x_values, y_margin1, '--', label="Margin")
plt.plot(x_values, y_margin2, '--')

plt.scatter(
    model.support_vectors_[:, 0],
    model.support_vectors_[:, 1],
    s=100,
    facecolors='none',
    edgecolors='black',
    label="Support Vectors"
)

plt.xlabel("Sepal Length")
plt.ylabel("Sepal Width")
plt.title("Linear SVM - Setosa vs Non-Setosa")
plt.legend()
plt.grid()
plt.show()
```

Polynomial regression